

Lab 1.1: Basic LoRaWAN Join

Course 1: LoRa & LoRaWAN Fundamentals

USP Training Series

1 Lab Overview

Lab Objectives

By the end of this lab, you will be able to:

- Configure LoRaWAN credentials (DevEUI, JoinEUI, AppKey)
- Perform OTAA (Over-The-Air Activation) join procedure
- Understand the join accept message
- Monitor join status through serial console
- Verify device registration on network server

Prerequisites

Required Knowledge:

- Basic understanding of LoRaWAN architecture
- Familiarity with Zephyr RTOS build system
- Access to LoRaWAN network server (TTN, ChirpStack, etc.)

Hardware Required:

- Xiao nRF54L15 development board
- LR1120 LoRa shield
- USB cable for programming and serial console
- LoRaWAN gateway within range

Software Required:

- Zephyr SDK installed
- West tool configured
- Serial terminal (screen, minicom, or PuTTY)

2 Lab Duration

Estimated Time: 45 minutes

- Setup and configuration: 15 minutes
- Implementation: 20 minutes
- Testing and verification: 10 minutes

3 Background

3.1 OTAA Join Procedure

LoRaWAN supports two activation methods:

- **OTAA (Over-The-Air Activation):** Device performs join procedure to obtain session keys
- **ABP (Activation By Personalization):** Session keys pre-configured in device

OTAA is the recommended method for production deployments because:

- More secure (keys derived from join procedure)
- Supports key rotation
- Enables device mobility between networks

3.2 Join Request Message

The join request contains:

- **JoinEUI** (formerly AppEUI): 64-bit identifier for join server
- **DevEUI**: 64-bit globally unique device identifier
- **DevNonce**: 16-bit random value (prevents replay attacks)

The message is encrypted with the AppKey.

3.3 Join Accept Message

Upon successful join, the network server responds with:

- **JoinNonce**: Random value from network
- **NetID**: Network identifier
- **DevAddr**: Device address (for this session)
- **DLSettings**: Downlink settings (RX1 delay, RX2 data rate)
- **RXDelay**: RX window timing

From this, the device derives:

- **NwkSKey**: Network session key (for MAC commands)
- **AppSKey**: Application session key (for payload encryption)

4 Lab Instructions

4.1 Step 1: Register Device on Network Server

1. Log in to your LoRaWAN network server (e.g., The Things Network)
2. Create a new application (if not already exists)
3. Add a new end device:
 - Activation mode: OTAA
 - LoRaWAN version: 1.0.4
 - Regional Parameters: EU868 (or your region)
 - DevEUI: Copy from device label or generate
 - JoinEUI: Use your application's JoinEUI
 - AppKey: Generate a new 128-bit key
4. Save the credentials for the next step

4.2 Step 2: Configure Device Credentials

Navigate to your project directory and edit the configuration file:

```
1 cd samples/lora/lorawan_hello
```

Create or edit `prj.conf`:

```
1 # LoRaWAN Configuration
2 CONFIG_LORAWAN=y
3 CONFIG_LORAWAN_REGION_EU868=y
4 CONFIG_LORAWAN_ACTIVATION_OTAA=y
5
6 # Enable logging
7 CONFIG_LOG=y
8 CONFIG_LORAWAN_LOG_LEVEL_DBG=y
```

Edit `src/main.c` to add your credentials (see Solution section).

4.3 Step 3: Build the Firmware

```
1 west build -b xiao_nrf54l15 samples/lora/lorawan_hello
```

4.4 Step 4: Flash the Device

```
1 west flash
```

4.5 Step 5: Monitor Serial Output

Open a serial terminal:

```
1 # Linux/macOS
2 screen /dev/ttyUSB0 115200
3
4 # Or use minicom
5 minicom -D /dev/ttyUSB0 -b 115200
```

4.6 Step 6: Observe Join Procedure

You should see output similar to:

```

1 [00:00:00.100,000] <inf> lorawan: LoRaWAN stack initialized
2 [00:00:00.150,000] <inf> lorawan: Starting OTAA join procedure
3 [00:00:05.230,000] <inf> lorawan: Join request sent
4 [00:00:10.450,000] <inf> lorawan: Join accept received
5 [00:00:10.451,000] <inf> lorawan: Network joined successfully!
6 [00:00:10.452,000] <inf> lorawan: DevAddr: 0x260B1234

```

4.7 Step 7: Verify on Network Server

1. Return to network server console
2. Navigate to your device page
3. Check "Live Data" or "Events" tab
4. Verify join request was received
5. Confirm join accept was sent
6. Device should show as "Active" or "Online"

5 Solution

Solution

5.1 Complete main.c Implementation

```

1 #include <zephyr/kernel.h>
2 #include <zephyr/device.h>
3 #include <zephyr/logging/log.h>
4 #include <zephyr/lorawan/lorawan.h>
5
6 LOG_MODULE_REGISTER(lab_1_1, LOG_LEVEL_INF);
7
8 /* LoRaWAN Credentials - REPLACE WITH YOUR OWN */
9 #define LORAWAN_DEV_EUI { 0x00, 0x00, 0x00, 0x00, \
10                          0x00, 0x00, 0x00, 0x01 }
11 #define LORAWAN_JOIN_EUI { 0x00, 0x00, 0x00, 0x00, \
12                             0x00, 0x00, 0x00, 0x00 }
13 #define LORAWAN_APP_KEY { 0x00, 0x00, 0x00, 0x00, \
14                             0x00, 0x00, 0x00, 0x00, \
15                             0x00, 0x00, 0x00, 0x00, \
16                             0x00, 0x00, 0x00, 0x00 }
17
18 static uint8_t dev_eui[] = LORAWAN_DEV_EUI;
19 static uint8_t join_eui[] = LORAWAN_JOIN_EUI;
20 static uint8_t app_key[] = LORAWAN_APP_KEY;
21
22 static struct lorawan_join_config join_cfg = {
23     .mode = LORAWAN_ACT_OTAA,
24     .dev_eui = dev_eui,
25     .otaa = {
26         .join_eui = join_eui,
27         .app_key = app_key,

```

```

28     .nwk_key = app_key, // For LoRaWAN 1.0.x
29 }
30 };
31
32 /* LoRaWAN downlink callback */
33 static void dl_callback(uint8_t port, bool data_pending,
34                        int16_t rssi, int8_t snr,
35                        uint8_t len, const uint8_t *data)
36 {
37     LOG_INF("Downlink received:");
38     LOG_INF("  Port: %d", port);
39     LOG_INF("  RSSI: %d dBm", rssi);
40     LOG_INF("  SNR: %d dB", snr);
41     LOG_INF("  Length: %d bytes", len);
42
43     if (len > 0) {
44         LOG_HEXDUMP_INF(data, len, "Payload");
45     }
46 }
47
48 /* LoRaWAN datarate changed callback */
49 static void dr_changed_callback(enum lorawan_datarate dr)
50 {
51     LOG_INF("Datarate changed to: DR%d", dr);
52 }
53
54 int main(void)
55 {
56     int ret;
57     const struct device *lora_dev;
58
59     LOG_INF("=====");
60     LOG_INF("Lab 1.1: Basic LoRaWAN Join");
61     LOG_INF("=====");
62
63     /* Get LoRa device */
64     lora_dev = DEVICE_DT_GET(DT_ALIAS(lora0));
65     if (!device_is_ready(lora_dev)) {
66         LOG_ERR("LoRa device not ready");
67         return -1;
68     }
69
70     LOG_INF("LoRa device ready");
71
72     /* Initialize LoRaWAN stack */
73     ret = lorawan_start();
74     if (ret < 0) {
75         LOG_ERR("Failed to start LoRaWAN stack: %d", ret);
76         return ret;
77     }
78
79     LOG_INF("LoRaWAN stack started");
80
81     /* Register callbacks */
82     struct lorawan_downlink_cb downlink_cb = {
83         .port = LW_RECV_PORT_ANY,

```

```

84     .cb = dl_callback
85 };
86 lorawan_register_downlink_callback(&downlink_cb);
87 lorawan_register_dr_changed_callback(dr_changed_callback);
88
89 LOG_INF("Callbacks registered");
90
91 /* Perform OTAA join */
92 LOG_INF("Starting OTAA join procedure...");
93 ret = lorawan_join(&join_cfg);
94 if (ret < 0) {
95     LOG_ERR("Join failed: %d", ret);
96     return ret;
97 }
98
99 LOG_INF("Join request sent, waiting for accept...");
100
101 /* Wait for join to complete */
102 while (!lorawan_is_joined()) {
103     k_sleep(K_SECONDS(1));
104 }
105
106 LOG_INF("=====");
107 LOG_INF("SUCCESS: Network joined!");
108 LOG_INF("=====");
109
110 /* Get and display DevAddr */
111 uint32_t dev_addr;
112 lorawan_get_dev_addr(&dev_addr);
113 LOG_INF("Assigned DevAddr: 0x%08X", dev_addr);
114
115 /* Keep application running */
116 while (1) {
117     k_sleep(K_SECONDS(60));
118     LOG_INF("Device active, joined to network");
119 }
120
121 return 0;
122 }

```

5.2 Build Configuration (prj.conf)

```

1  # LoRaWAN Configuration
2  CONFIG_LORAWAN=y
3  CONFIG_LORAWAN_REGION_EU868=y
4  CONFIG_LORAWAN_ACTIVATION_OTAA=y
5
6  # Enable logging
7  CONFIG_LOG=y
8  CONFIG_LORAWAN_LOG_LEVEL_DBG=y
9  CONFIG_LOG_BUFFER_SIZE=2048
10
11 # Network stack
12 CONFIG_NETWORKING=y
13 CONFIG_NET_L2_LORAWAN=y

```

```

14
15 # Increase stack sizes
16 CONFIG_MAIN_STACK_SIZE=2048
17 CONFIG_SYSTEM_WORKQUEUE_STACK_SIZE=2048

```

6 Expected Output

When the device successfully joins, you should see:

```

1 =====
2 Lab 1.1: Basic LoRaWAN Join
3 =====
4 [00:00:00.100] <inf> lab_1_1: LoRa device ready
5 [00:00:00.105] <inf> lab_1_1: LoRaWAN stack started
6 [00:00:00.110] <inf> lab_1_1: Callbacks registered
7 [00:00:00.115] <inf> lab_1_1: Starting OTAA join procedure...
8 [00:00:00.120] <inf> lab_1_1: Join request sent, waiting...
9 [00:00:05.234] <inf> lorawan: TX: Join Request
10 [00:00:05.235] <inf> lorawan: Frequency: 868.1 MHz
11 [00:00:05.236] <inf> lorawan: DR: SF12BW125
12 [00:00:10.567] <inf> lorawan: RX: Join Accept
13 [00:00:10.568] <inf> lorawan: Network joined
14 =====
15 SUCCESS: Network joined!
16 =====
17 [00:00:10.570] <inf> lab_1_1: Assigned DevAddr: 0x260B1234
18 [00:01:10.570] <inf> lab_1_1: Device active, joined to network

```

7 Troubleshooting

7.1 Join Request Not Received by Gateway

Warning

Symptoms: No join request appears on network server

Possible Causes:

- Gateway out of range
- Incorrect frequency plan (EU868 vs US915)
- Antenna not connected
- Radio not configured properly

Solutions:

- Check gateway status and location
- Verify regional parameters match gateway
- Ensure antenna is securely connected
- Check device tree configuration for radio

7.2 Join Accept Not Received

Warning

Symptoms: Join request sent but no accept received

Possible Causes:

- Incorrect credentials (DevEUI, JoinEUI, AppKey)
- RX window timing issues
- Device not registered on network server

Solutions:

- Double-check all credentials match network server
- Verify device is registered and enabled
- Check RX window configuration
- Ensure AppKey is correct (case-sensitive)

7.3 Build Errors

Warning

Common Build Issues:

- Missing `CONFIG_LORAWAN=y` in `prj.conf`
- Incorrect board name
- Missing Zephyr modules

Solutions:

- Clean build: `west build -t pristine`
- Update Zephyr: `west update`
- Check board support: `west boards | grep xiao`

8 Deliverables

Upon completion of this lab, you should submit:

1. Source Code

- Complete `main.c` with your credentials
- `prj.conf` configuration file

2. Serial Console Log

- Full output showing successful join
- Include timestamps
- Highlight DevAddr assigned

3. Network Server Screenshot

- Device showing as "Active" or "Online"
- Join request/accept visible in events
- DevAddr matches console output

4. Lab Report

- Brief description of procedure followed
- Any issues encountered and how resolved
- Time taken for join procedure
- RSSI and SNR values observed

9 Additional Challenges

9.1 Challenge 1: Analyze Join Timing

Measure the time from join request to join accept:

- Add timestamp logging
- Calculate total join time
- Identify RX1 vs RX2 window used

9.2 Challenge 2: Multiple Join Attempts

Modify the code to handle join failures:

- Implement exponential backoff
- Retry up to 3 times
- Log each attempt

9.3 Challenge 3: Display Join Parameters

Extract and display join accept parameters:

- RX1 delay
- RX2 data rate
- RX2 frequency
- CFList (if present)

10 References

- LoRaWAN 1.0.4 Specification (LoRa Alliance)
- LoRaWAN Regional Parameters (RP002-1.0.4)
- Zephyr LoRaWAN API Documentation
- Course 1 Presentation Slides

11 Next Steps

After completing this lab, you should proceed to:

- **Lab 1.2:** Send and receive messages
- **Lab 1.3:** Implement confirmed uplinks
- **Lab 1.4:** Downlink handling and Class A operation